

LR objektinio programavimo uzduotis 3

Generated by Doxygen 1.13.2



|                                                                                                                           |           |
|---------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1 Projekto paleidimo instrukcija</b>                                                                                   | <b>1</b>  |
| 1.1 Testai                                                                                                                | 1         |
| 1.1.1 Strategija 1, konteineris: vector, duomenu tipas: klase                                                             | 1         |
| 1.1.2 Strategija 1, konteineris: vector, duomenu tipas: struktura                                                         | 1         |
| 1.1.3 Optimizacija: O1                                                                                                    | 1         |
| 1.1.3.1 Strategija 1, konteineris: vector, duomenu tipas: struktura                                                       | 1         |
| 1.1.3.2 Strategija 1, konteineris: vector, duomenu tipas: klase                                                           | 2         |
| 1.1.4 Optimizacija: O2                                                                                                    | 2         |
| 1.1.4.1 Strategija 1, konteineris: vector, duomenu tipas: struktura                                                       | 2         |
| 1.1.4.2 Strategija 1, konteineris: vector, duomenu tipas: klase                                                           | 2         |
| 1.1.5 Optimizacija: O3                                                                                                    | 2         |
| 1.1.5.1 Strategija 1, konteineris: vector, duomenu tipas: struktura                                                       | 2         |
| 1.1.5.2 Strategija 1, konteineris: vector, duomenu tipas: klase                                                           | 3         |
| 1.1.6 Failu dydžiai:                                                                                                      | 3         |
| 1.1.6.1 su struktūromis                                                                                                   | 3         |
| 1.1.6.2 su klasėmis                                                                                                       | 3         |
| 1.2 "Rule of Five"                                                                                                        | 3         |
| 1.2.1 Kiekvieno metodo paaiškinimas                                                                                       | 3         |
| 1.2.1.1 1. Destruktorius                                                                                                  | 3         |
| 1.2.1.2 2. Kopijavimo konstruktorius                                                                                      | 4         |
| 1.2.1.3 3. Kopijavimo priskyrimo operatorius                                                                              | 4         |
| 1.2.1.4 4. Perkėlimo konstruktorius                                                                                       | 4         |
| 1.2.1.5 5. Perkėlimo priskyrimo operatorius                                                                               | 5         |
| 1.3 Operatorių perdengimas (<< ir >>)                                                                                     | 5         |
| 1.3.1 Kas tai yra?                                                                                                        | 5         |
| 1.3.2 Perdengimų paaiškinimai                                                                                             | 5         |
| 1.3.2.1 1. Įvesties operatorius >>                                                                                        | 5         |
| 1.3.2.2 2. Išvesties operatorius <<                                                                                       | 5         |
| 1.4 Santrauka                                                                                                             | 6         |
| 1.5 Abstrakčios klasės                                                                                                    | 6         |
| 1.5.1 Kas yra abstrakti klasė?                                                                                            | 6         |
| 1.5.2 Dabartinė <code>Zmogus</code> klasės implementacija                                                                 | 6         |
| 1.5.2.1 Komentarai apie implementaciją:                                                                                   | 6         |
| 1.5.3 Testai                                                                                                              | 6         |
| 1.5.3.1 <code>std::vector</code> ir mano <code>Vektorius</code> <code>push_back()</code> greitis ir persikirstymo greitis | 6         |
| 1.5.3.2 <code>std::vector</code> ir mano <code>Vektorius</code> failo generavimas                                         | 7         |
| 1.5.3.3 <code>std::vector</code> ir mano <code>Vektorius</code> failo <code>read/sort/divide</code> greitis               | 7         |
| <b>2 Hierarchical Index</b>                                                                                               | <b>9</b>  |
| 2.1 Class Hierarchy                                                                                                       | 9         |
| <b>3 Class Index</b>                                                                                                      | <b>11</b> |
| 3.1 Class List                                                                                                            | 11        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>4 File Index</b>                                               | <b>13</b> |
| 4.1 File List . . . . .                                           | 13        |
| <b>5 Class Documentation</b>                                      | <b>15</b> |
| 5.1 Studentas Class Reference . . . . .                           | 15        |
| 5.1.1 Member Function Documentation . . . . .                     | 15        |
| 5.1.1.1 prisistatymas() . . . . .                                 | 15        |
| 5.2 Vektorius< ElementoTipas > Class Template Reference . . . . . | 16        |
| 5.3 Zmogus Class Reference . . . . .                              | 16        |
| <b>6 File Documentation</b>                                       | <b>17</b> |
| 6.1 funkcijos.h . . . . .                                         | 17        |
| 6.2 manoVektorius.h . . . . .                                     | 21        |
| 6.3 pagalbines.h . . . . .                                        | 24        |
| <b>Index</b>                                                      | <b>27</b> |

# Chapter 1

## Projekto paleidimo instrukcija

1. Naudodami git, galite klonuoti si projekta su komanda `git clone https://github.com/Liveta/Objektinis-Programavimas.git`
2. Sukurkite projekto .exe faila, paleisdami `make` komanda projekto direktorijoje
3. Paleiskite projekta, paleisdami `./vektoriai` komanda

### 1.1 Testai

#### 1.1.1 Strategija 1, konteineris: vector, duomenu tipas: klase

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0187384 s     | 0.0025954 s      | 0.001142 s        |
| 10000           | 0.0464928 s     | 0.0173884 s      | 0.0080451 s       |
| 100000          | 0.554703 s      | 0.299301 s       | 0.076983 s        |
| 1000000         | 3.4679 s        | 2.6752 s         | 0.528214 s        |
| 10000000        | 3.16855 s       | 2.72026 s        | 0.54384 s         |

#### 1.1.2 Strategija 1, konteineris: vector, duomenu tipas: struktura

| Studentu kiekis | Skaitymas   | Rikiavimas  | Skirstymas  |
|-----------------|-------------|-------------|-------------|
| 1000            | 0.0067999 s | 0.000899 s  | 0.0004967 s |
| 10000           | 0.0451155 s | 0.0144912 s | 0.0055211 s |
| 100000          | 0.379105 s  | 0.166723 s  | 0.0503665 s |
| 1000000         | 3.25661 s   | 1.66952 s   | 0.473 s     |
| 10000000        | 33.9667 s   | 22.6545 s   | 6.74625 s   |

#### 1.1.3 Optimizacija: O1

##### 1.1.3.1 Strategija 1, konteineris: vector, duomenu tipas: struktura

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0064206 s     | 0.0006721 s      | 0.0004453 s       |
| 10000           | 0.0630172 s     | 0.0037798 s      | 0.004321 s        |
| 100000          | 0.491349 s      | 0.0531723 s      | 0.0415576 s       |
| 1000000         | 3.50831 s       | 0.442264 s       | 0.323002 s        |
| 10000000        | 3.51897 s       | 0.351057 s       | 0.288263 s        |

#### 1.1.3.2 Strategija 1, konteineris: vector, duomenu tipas: klase

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0041683 s     | 0.0007185 s      | 0.0004925 s       |
| 10000           | 0.0346105 s     | 0.0063844 s      | 0.00333 s         |
| 100000          | 0.225924 s      | 0.0731626 s      | 0.0277155 s       |
| 1000000         | 2.47328 s       | 1.06697 s        | 0.339018 s        |
| 10000000        | 3.62191 s       | 1.5846 s         | 0.36761 s         |

### 1.1.4 Optimizacija: O2

#### 1.1.4.1 Strategija 1, konteineris: vector, duomenu tipas: struktura

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0101266 s     | 0.0005713 s      | 0.0006246 s       |
| 10000           | 0.0275356 s     | 0.0015958 s      | 0.0026655 s       |
| 100000          | 0.239583 s      | 0.0266187 s      | 0.0226893 s       |
| 1000000         | 2.20875 s       | 0.305261 s       | 0.23545 s         |
| 10000000        | 4.35964 s       | 0.850409 s       | 0.433324 s        |

#### 1.1.4.2 Strategija 1, konteineris: vector, duomenu tipas: klase

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0032936 s     | 0.0005334 s      | 0.0006714 s       |
| 10000           | 0.0493008 s     | 0.0114337 s      | 0.0065617 s       |
| 100000          | 0.447514 s      | 0.152694 s       | 0.0603926 s       |
| 1000000         | 4.67117 s       | 1.98655 s        | 0.563572 s        |
| 10000000        | 4.95353 s       | 1.89983 s        | 0.423116 s        |

### 1.1.5 Optimizacija: O3

#### 1.1.5.1 Strategija 1, konteineris: vector, duomenu tipas: struktura

| Studentu kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0075853 s     | 0.0005195 s      | 0.0006241 s       |

|          |             |             |             |
|----------|-------------|-------------|-------------|
| 10000    | 0.0287904 s | 0.0025298 s | 0.0021945 s |
| 100000   | 0.235563 s  | 0.0220492 s | 0.0218198 s |
| 1000000  | 2.67391 s   | 0.281722 s  | 0.254976 s  |
| 10000000 | 3.97697 s   | 0.518433 s  | 0.417878 s  |

### 1.1.5.2 Strategija 1, konteineris: vector, duomenų tipas: klase

| Studentų kiekis | failo skaitymas | failo rikiavimas | failo filtravimas |
|-----------------|-----------------|------------------|-------------------|
| 1000            | 0.0048808 s     | 0.0008823 s      | 0.0006448 s       |
| 10000           | 0.0644253 s     | 0.0124485 s      | 0.0111525 s       |
| 100000          | 0.455858 s      | 0.132958 s       | 0.0530912 s       |
| 1000000         | 4.73698 s       | 1.63243 s        | 0.424595 s        |
| 10000000        | 4.99333 s       | 1.27652 s        | 0.339926 s        |

## 1.1.6 Failų dydžiai:

### 1.1.6.1 su struktūromis

vektoriai.exe : 622KB vektoriai\_O1.exe : 319KB vektoriai\_O2.exe : 298KB vektoriai\_O3.exe : 294KB

### 1.1.6.2 su klasėmis

vektoriai.exe : 712KB vektoriai\_O1.exe : 349KB vektoriai\_O2.exe : 317KB vektoriai\_O3.exe : 310KB

## 1.2 "Rule of Five"

### Kas yra "Rule of Five"?

C++ kalboje, jei klasė valdo išteklius (pvz., dinaminę atmintį, failus ar pan.), reikia apibrėžti penkis specialius metodus:

1. Destruktorius
2. Kopijavimo konstruktorius
3. Kopijavimo priskyrimo operatorius
4. Perkėlimo konstruktorius
5. Perkėlimo priskyrimo operatorius

### 1.2.1 Kiekvieno metodo paaiškinimas

#### 1.2.1.1 1. Destruktorius

##### Paskirtis:

- Atlaisvina išteklius, kai objektas sunaikinamas.

##### Pavyzdys:

```
Studentas::~Studentas() {
    vardas.clear();
    pavarde.clear();
    pazymiai.clear();
}
```

### 1.2.1.2 2. Kopijavimo konstruktorius

#### Paskirtis:

- Sukuria naują objektą kaip tikslią kito objekto kopiją.

#### Panaudojimas:

- Naudojamas, kai reikia nukopijuoti objektą, kad būtų sukurtas nepriklausomas jo dublikatas.

#### Pavyzdys:

```
Studentas::Studentas(const Studentas& originalas) {  
    pavarde = originalas.pavarde;  
    vardas = originalas.vardas;  
    pazymiai = originalas.pazymiai;  
    egzamino_pazymys = originalas egzamino_pazymys;  
    vidurkis = originalas.vidurkis;  
    mediana = originalas.mediana;  
}
```

### 1.2.1.3 3. Kopijavimo priskyrimo operatorius

#### Paskirtis:

- Pakeičia esamo objekto turinį kitu objektu.

#### Panaudojimas:

- `a = b;` kai abu objektai jau egzistuoja.

#### Pavyzdys:

```
Studentas& Studentas::operator=(const Studentas& originalas) {  
    if (this != &originalas) {  
        pavarde = originalas.pavarde;  
        vardas = originalas.vardas;  
        pazymiai = originalas.pazymiai;  
        egzamino_pazymys = originalas egzamino_pazymys;  
        vidurkis = originalas.vidurkis;  
        mediana = originalas.mediana;  
    }  
    return *this;  
}
```

### 1.2.1.4 4. Perkėlimo konstruktorius

#### Paskirtis:

- Sukuria naują objektą, "pavogdamas" išteklius iš kito objekto.

#### Panaudojimas:

- Efektyvus darbas su laikiniais objektais.

#### Pavyzdys:

```
Studentas::Studentas(Studentas&& originalas) noexcept {  
    pavarde = move(originalas.pavarde);  
    vardas = move(originalas.vardas);  
    pazymiai = move(originalas.pazymiai);  
    egzamino_pazymys = originalas egzamino_pazymys;  
    vidurkis = originalas.vidurkis;  
    mediana = originalas.mediana;  
  
    originalas egzamino_pazymys = 0;  
    originalas.vidurkis = 0;  
    originalas.mediana = 0;  
}
```



### 1.2.1.5 5. Perkėlimo priskyrimo operatorius

#### Paskirtis:

- Pakeičia objekto turinį, "pavogdamas" iš kito objekto išteklius.

#### Panaudojimas:

- `a = move(b);`

#### Pavyzdys:

```
Studentas& Studentas::operator=(Studentas&& originalas) noexcept {
    if (this != &originalas) {
        pavarde = move(originalas.pavarde);
        vardas = move(originalas.vardas);
        pazymiai = move(originalas.pazymiai);
        egzamino_pazymys = originalas.egzamino_pazymys;
        vidurkis = originalas.vidurkis;
        mediana = originalas.mediana;

        originalas.egzamino_pazymys = 0;
        originalas.vidurkis = 0;
        originalas.mediana = 0;
    }
    return *this;
}
```

---

## 1.3 Operatorių perdengimas (<< ir >>)

### 1.3.1 Kas tai yra?

Operatorių perdengimas leidžia nustatyti, kaip objektai yra skaitomi iš srauto (>>) ir spausdinami į srautą (<<). Tai leidžia naudoti paprastą sintaksę kaip `cout << objektas; arba cin >> objektas;`.

---

### 1.3.2 Perdengimų paaiškinimai

#### 1.3.2.1 1. Įvesties operatorius >>

##### Paskirtis:

- Nuskaito `Studentas` objekto duomenis iš vartotojo arba failo.

##### Pavyzdys:

```
istream& operator>>(istream& isvesties_vieta, Studentas& studentas) {
    cout << "Įveskite vardą: ";
    isvesties_vieta >> studentas.vardas;

    cout << "Įveskite pavardę: ";
    isvesties_vieta >> studentas.pavarde;

    cout << "Įveskite pažymių kiekį: ";
    int kiekis;
    isvesties_vieta >> kiekis;

    studentas.pazymiai.clear();
    cout << "Įveskite pažymius: ";
    for (int i = 0; i < kiekis; ++i) {
        int pazymys;
        isvesties_vieta >> pazymys;
        studentas.pazymiai.push_back(pazymys);
    }

    cout << "Įveskite egzamino pažymį: ";
    isvesties_vieta >> studentas.egzamino_pazymys;

    studentas.vidurkio_skaiciavimas();
    studentas.medianos_skaiciavimas();

    return isvesties_vieta;
}
```

---

#### 1.3.2.2 2. Išvesties operatorius <<

##### Paskirtis:

- Išveda `Studentas` objekto duomenis į ekraną arba failą.

#### Pavyzdys:

```
ostream& operator<<(ostream& isvesties_vieta, const Studentas& studentas) {
    isvesties_vieta << studentas vardas << " " << studentas pavarde << " | Pažymiai: ";
    for (int pazymys : studentas.pazymiai) {
        isvesties_vieta << pazymys << " ";
    }
    isvesties_vieta << " | Egzaminas: " << studentas.egzamino_pazymys;
    isvesties_vieta << " | Vidurkis: " << studentas.vidurkis;
    isvesties_vieta << " | Mediana: " << studentas.mediana;
    return isvesties_vieta;
}
```

## 1.4 Santrauka

| Koncepcija                                  | Paskirtis                                | Pavyzdys                              |
|---------------------------------------------|------------------------------------------|---------------------------------------|
| Destruktorius                               | Atlaisvina išteklius sunaikinant objektą | <code>~Studentas()</code>             |
| Kopijavimo konstruktorius                   | Sukuria naują kopiją                     | <code>Studentas b = a;</code>         |
| Kopijavimo priskyrimas                      | Pakeičia objekto turinį kitu             | <code>b = a;</code>                   |
| Perkėlimo konstruktorius                    | Pavogia išteklius iš kito objekto        | <code>Studentas b = move(a);</code>   |
| Perkėlimo priskyrimas                       | Pavogia išteklius per priskyrimą         | <code>b = move(a);</code>             |
| Išvesties operatorius <code>&lt;&lt;</code> | Išveda objektą į ekraną arba failą       | <code>cout &lt;&lt; studentas;</code> |
| Įvesties operatorius <code>&gt;&gt;</code>  | Nuskaityti objektą iš vartotojo ar failo | <code>cin &gt;&gt; studentas;</code>  |

## 1.5 Abstrakčios klasės

### 1.5.1 Kas yra abstrakti klasė?

Abstrakti klasė yra tokia klasė, kuri aprašo bendras sąvokas ir turi bent vieną abstraktų metodą (be implementacijos). Tokia klasė negali būti sukuriamą tiesiogiai, ji naudojama kaip bazė kitoms klasėms paveldėti ir konkretizuoti elgesį. Abstrakčios klasės tikslas - apibrėžti, kokias funkcijas turi įgyvendinti paveldėtos klasės, paliekant realizacijos detales joms pačioms.

### 1.5.2 Dabartinė Zmogus klasės implementacija

```
class Zmogus{
public:
    virtual void prisistatymas() = 0;
    virtual ~Zmogus() = default;
};
```

#### 1.5.2.1 Komentaras apie implementaciją:

- `prisistatymas()` yra **grynas virtualus metodas** (`= 0`), kuris reiškia, kad **kiekviena paveldinti klasė** privalo įgyvendinti savo prisistatymo logiką.
- `virtual ~Zmogus() = default;` garantuoja, kad sunaikinant išvestinės klasės objektą bus teisingai kviečiamas destruktoriaus.

Ši bazinė struktūra yra kaip pagrindas, leidžiantis kurti įvairias `Zmogus` klases, tokias kaip `Studentas`, `Darbuotojas` ir pan., priversdama jas implementuoti savo `prisistatymas()` metodo.

### 1.5.3 Testai

#### 1.5.3.1 `std::vector` ir mano `Vektorius` `push_back()` greitis ir perskirstymo greitis

| Studentu kiekis | <code>std::vector</code> laikas | Mano <b>Vektorius</b> laikas | <code>std::vector</code> perskirstymas | Mano <b>Vektorius</b> perskirstymas |
|-----------------|---------------------------------|------------------------------|----------------------------------------|-------------------------------------|
| 10000           | 0 s                             | 0 s                          | 15                                     | 10                                  |

| Studentu kiekis | std::vector laikas | Mano Vektorius laikas | std::vector perskirstymas | Mano Vektorius perskirstymas |
|-----------------|--------------------|-----------------------|---------------------------|------------------------------|
| 100000          | 0.001794 s         | 0.001915 s            | 18                        | 14                           |
| 1000000         | 0.021624 s         | 0.016285 s            | 21                        | 17                           |
| 10000000        | 0.201588 s         | 0.134709 s            | 25                        | 20                           |
| 100000000       | 2.1565s            | 1.79215s              | 28                        | 24                           |

### 1.5.3.2 std::vector ir mano Vektorius failo generavimas

| Studentu kiekis | std::vector laikas | Mano Vektorius laikas |
|-----------------|--------------------|-----------------------|
| 1000            | 0.014633s          | 0.013252s             |
| 10000           | 0.137478s          | 0.104858s             |
| 100000          | 0.968856s          | 1.06672s              |
| 1000000         | 9.64237s           | 10.2498s              |
| 10000000        | 117.642s           | 106.34815s            |

### 1.5.3.3 std::vector ir mano Vektorius failo read/sort/divide greitis

| std::vector       | 1000 studentai | 10000 studentai | 100000 studentai | 1000000 studentai | 10000000 studentai |
|-------------------|----------------|-----------------|------------------|-------------------|--------------------|
| Failo skaitymas↔: | 0.08294s       | 0.165984s       | 1.542088s        | 15.41559s         | 165.246s           |
| Rūšiavimas:       | 0.005981s      | 0.0560194s      | 0.5255879s       | 6.19015s          | 60.56389s          |
| Filtravimas:      | 0.0019701s     | 0.0088287s      | 0.0874609s       | 1.12756s          | 11.3952s           |

| Mano Vektorius    | 1000 studentai | 10000 studentai | 100000 studentai | 1000000 studentai | 10000000 studentai |
|-------------------|----------------|-----------------|------------------|-------------------|--------------------|
| Failo skaitymas↔: | 0.0089653s     | 0.08539s        | 0.8317273s       | 8.32632s          | 84.6576s           |
| Rūšiavimas:       | 0.003841s      | 0.0927s         | 0.4375459s       | 5.6459764s        | 58.65171s          |
| Filtravimas:      | 0.000958s      | 0.01088s        | 0.099862s        | 1.096143s         | 11.39709s          |



## Chapter 2

# Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                                      |    |
|--------------------------------------|----|
| Vektorius< ElementoTipas > . . . . . | 16 |
| Zmogus . . . . .                     | 16 |
| Studentas . . . . .                  | 15 |



## Chapter 3

# Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                            |    |
|------------------------------------------------------------|----|
| <a href="#">Studentas</a> . . . . .                        | 15 |
| <a href="#">Vektorius&lt; ElementoTipas &gt;</a> . . . . . | 16 |
| <a href="#">Zmogus</a> . . . . .                           | 16 |





## Chapter 4

# File Index

### 4.1 File List

Here is a list of all documented files with brief descriptions:

|                                           |    |
|-------------------------------------------|----|
| <a href="#">funkcijos.h</a> . . . . .     | 17 |
| <a href="#">manoVektorius.h</a> . . . . . | 21 |
| <a href="#">pagalbines.h</a> . . . . .    | 24 |

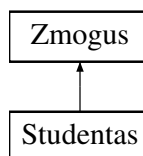


# Chapter 5

## Class Documentation

### 5.1 Studentas Class Reference

Inheritance diagram for Studentas:



#### Public Member Functions

- **Studentas** (string vardass, string pavardee, [Vektorius](#)< int > pazymiai, int egzamino\_pazymyss)
- **Studentas** (const [Studentas](#) &originalas)
- [Studentas](#) & **operator=** (const [Studentas](#) &originalas)
- **Studentas** ([Studentas](#) &&originalas) noexcept
- [Studentas](#) & **operator=** ([Studentas](#) &&originalas) noexcept
- void [prisistatymas](#) () override
- void **vidurkio\_skaiciavimas** ()
- void **medianos\_skaiciavimas** ()

#### Public Attributes

- string **pavarde**
- string **vardas**
- [Vektorius](#)< int > **pazymiai**
- int **egzamino\_pazymys**
- double **vidurkis**
- double **mediana**

#### Friends

- ostream & **operator**<< (ostream &isvesties\_vieta, const [Studentas](#) &studentas)
- istream & **operator**>> (istream &isvesties\_vieta, [Studentas](#) &studentas)

#### 5.1.1 Member Function Documentation

##### 5.1.1.1 prisistatymas()

```
void Studentas::prisistatymas () [override], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- funkcijos.h
- funkcijos.cpp

## 5.2 Vektorius< ElementoTipas > Class Template Reference

### Public Member Functions

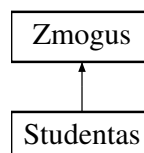
- **Vektorius** (const [Vektorius](#) &pagrindinis)
- **Vektorius** & **operator=** (const [Vektorius](#) &pagrindinis)
- **Vektorius** ([Vektorius](#) &&pagrindinis) noexcept
- **Vektorius** & **operator=** ([Vektorius](#) &&pagrindinis) noexcept
- **Vektorius** (ElementoTipas \*first, ElementoTipas \*last)
- ElementoTipas & **operator[]** (size\_t index)
- const ElementoTipas & **operator[]** (size\_t index) const
- size\_t **size** () const
- size\_t **capacity** () const
- ElementoTipas \* **begin** ()
- ElementoTipas \* **end** ()
- const ElementoTipas \* **begin** () const
- const ElementoTipas \* **end** () const
- ElementoTipas & **front** ()
- ElementoTipas & **back** ()
- const ElementoTipas & **front** () const
- const ElementoTipas & **back** () const
- void **push\_back** (const ElementoTipas &elementas)
- void **pop\_back** ()
- bool **empty** () const
- ElementoTipas & **at** (size\_t index)
- void **clear** ()
- void **shrink\_to\_fit** ()
- void **reserve** (size\_t naujaTalpa)
- void **erase** (size\_t index)
- ElementoTipas \* **erase** (ElementoTipas \*it)

The documentation for this class was generated from the following file:

- manoVektorius.h

## 5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



### Public Member Functions

- virtual void **prisistatymas** ()=0

The documentation for this class was generated from the following file:

- funkcijos.h

# Chapter 6

## File Documentation

### 6.1 funkcijos.h

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "pagalbines.h"
00005 #include "manoVektorius.h"
00006
00007 class Zmogus{
00008     public:
00009         virtual void prisistatymas() = 0;
00010         virtual ~Zmogus() = default;
00011 };
00012
00013 class Studentas : public Zmogus{
00014     public:
00015         string pavarde;
00016         string vardas;
00017         Vektorius<int> pazymiai;
00018         int egzamino_pazymys;
00019         double vidurkis;
00020         double mediana;
00021
00022         Studentas(string vardas, string pavarde, Vektorius<int> pazymiai, int egzamino_pazymys);
00023         Studentas(const Studentas& originalas);
00024         Studentas& operator=(const Studentas& originalas);
00025         Studentas(Studentas&& originalas) noexcept;
00026         Studentas& operator=(Studentas&& originalas) noexcept;
00027         Studentas();
00028         ~Studentas();
00029         friend ostream& operator<<(ostream& isvesties_vieta, const Studentas& studentas);
00030         friend istream& operator>>(istream& isvesties_vieta, Studentas& studentas);
00031         void prisistatymas() override;
00032         void vidurkio_skaiciavimas();
00033         void medianos_skaiciavimas();
00034 };
00035
00036 extern int studentu_kiekis;
00037 extern Vektorius<string> vardai;
00038 extern Vektorius<string> pavardes;
00039 extern Vektorius<Studentas> studentai;
00040 extern Vektorius<double> operaciju_laikai;
00041 extern int operaciju_kiekis;
00042
00043 void failo_su_studentais_generavimas();
00044 int pasirinkimo_pavertimas_i_reiksme(int failo_generavimo_pasirinkimas);
00045 void failo_generavimas(int studentu_kiekis, int pazymiu_kiekis);
00046 int vieno_pazymio_sugeneravimas();
00047
00048 void studento_duomenu_gavimas(Vektorius<Studentas> &studentai);
00049 int studento_duomenu_surasymo_pasirinkimas();
00050 double laiku_vidurkio_skaiciavimas(Vektorius<double> &operaciju_laikai);
00051
00052 void studento_duomenu_printinimas(Vektorius<Studentas> &studentai);
00053 void studento_duomenu_rikiavimas(Vektorius<Studentas> &studentai);
00054 bool rikiuoti_pagal_varda(Studentas &a, Studentas &b);
00055 bool rikiuoti_pagal_pavarde(Studentas &a, Studentas &b);
00056 bool rikiuoti_pagal_vidurkis(Studentas &a, Studentas &b);
00057 bool rikiuoti_pagal_mediana(Studentas &a, Studentas &b);
00058 bool baigiasi_su_txt(string failo_pavadinimas);
00059
00060 void studento_vardo_ir_pavardes_gavimas(Studentas &laikinas_studentas);
00061 void studento_pazymiu_gavimas(Studentas &laikinas_studentas);
```

```

00062 void studento_egzamino_pazymio_gavimas(Studentas &laikinas_studentas);
00063
00064 void tikrinimas_ar_pavyko_atidaryti_faila(string& failo_pavadinimas);
00065
00066 void studento_pazymiu_ir_egzaminu_generavimas(Studentas &laikinas_studentas);
00067 int gauk_kiek_pazymiu_sugeneruoti();
00068 void studento_vardo_ir_pavardes_generavimas(Studentas &laikinas_studentas);
00069
00070 void visi_duomenys_surasomi_ranka();
00071 void surasoma_ranka_isskyrus_pazymius();
00072 void viskas_generuojama_atsitiktinai();
00073
00074
00075 template <typename konteineris>
00076 void studentu_medianos_skaiciavimas(konteineris &studentai){
00077     for (auto it = studentai.begin(); it != studentai.end(); ++it){
00078         it->medianos_skaiciavimas();
00079     }
00080 }
00081 template <typename konteineris>
00082 void duomenu_surasymas_i_faila(konteineris studentai, string failo_pavadinimas){
00083     ofstream failas(failo_pavadinimas);
00084     failas << setw(18) << left << "Vardas" << setw(18) << left << "Pavarde" << setw(20) << left << "Galutinis
(Vid.)" << setw(20) << left << "Galutinis (Med.)" << endl;
00085     failas << "-----" << endl;
00086
00087     for (auto iteratorius = studentai.begin(); iteratorius != studentai.end(); ++iteratorius) {
00088         failas << std::setw(18) << std::left << iteratorius->vardas
00089             << std::setw(18) << std::left << iteratorius->pavarde
00090             << std::setw(20) << std::left << std::fixed << std::setprecision(2) <<
iteratorius->vidurkis
00091             << std::setw(20) << iteratorius->mediana
00092             << std::endl;
00093     }
00094     failas.close();
00095 }
00096
00097
00098 template <typename konteineris>
00099 void studento_vidurkio_skaiciavimas(konteineris &studentai){
00100
00101     for (auto iteratorius = studentai.begin(); iteratorius != studentai.end(); ++iteratorius) {
00102         iteratorius->vidurkio_skaiciavimas();
00103     }
00104 }
00105
00106 template <typename konteineris>
00107 void studento_duomenu_is_failo_susirasymas(konteineris&studentai, string& failo_pavadinimas){
00108     auto pradzia = std::chrono::high_resolution_clock::now();
00109
00110     ifstream failas(failo_pavadinimas);
00111     string antrastine_eilute;
00112     getline(failas, antrastine_eilute);
00113
00114     string eilute;
00115
00116     while(getline(failas, eilute)){
00117         Studentas laikinas_studentas;
00118         stringstream eil(eilute);
00119         eil >> laikinas_studentas.vardas >> laikinas_studentas.pavarde;
00120         int pazymys;
00121         while (eil >> pazymys){
00122             laikinas_studentas.pazymiai.push_back(pazymys);
00123         }
00124         laikinas_studentas.egzamino_pazymys = laikinas_studentas.pazymiai.back();
00125         laikinas_studentas.pazymiai.pop_back();
00126         studentai.push_back(laikinas_studentas);
00127     }
00128     operaciju_kiekis += 1;
00129
00130     auto pabaiga = std::chrono::high_resolution_clock::now();
00131     std::chrono::duration<double> trukme = pabaiga - pradzia;
00132     cout << endl << "duomenu is failo nuskaitymas uztruko: " << trukme.count() << " s" << endl;
00133     operaciju_laikai.push_back(trukme.count());
00134     failas.close();
00135 }
00136
00137 template <typename konteineris>
00138 void studento_duomenu_skaitymas_is_failo(konteineris& studentai){
00139     string failo_pavadinimas;
00140     cout << "Iveskite norimo nuskaityti failo pavadinima: " << endl;
00141     cin >> failo_pavadinimas;
00142
00143     while (true){
00144         try{
00145             tikrinimas_ar_pavyko_atidaryti_faila(failo_pavadinimas);
00146             break;

```

```

00147         }catch(exception &eroras){
00148             cout << eroras.what() << endl;
00149             cin >> failo_pavadinimas;
00150         }
00151     }
00152     cout << "Failas atidarytas sėkmingai!" << endl;
00153     studento_duomeniu_is_failo_susirasymas(studentai, failo_pavadinimas);
00154 }
00155 }
00156
00157 template <typename konteineris>
00158 void strategija_1(){
00159     konteineris visi;
00160     konteineris vargsiukai;
00161     konteineris kietiakai;
00162
00163     studento_duomeniu_skaitymas_is_failo(visi);
00164     studento_vidurkio_skaiciavimas(visi);
00165
00166     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00167
00168     if constexpr (is_same_v<konteineris, list<Studentas> >){
00169         visi.sort(rikiuoti_pagal_vidurkis);
00170     } else{
00171         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00172     }
00173
00174     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00175     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00176     cout << endl << "Duomeniu rikiavimas nuo didžiausio vidurkio iki mažiausio užtruko: " <<
00177         rikiavimo_trukme.count() << " s" << endl;
00178
00179     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00180
00181     for (auto it = visi.begin(); it != visi.end(); ++it) {
00182         if (it->vidurkis >= 5) {
00183             kietiakai.push_back(*it);
00184         } else {
00185             vargsiukai.push_back(*it);
00186         }
00187     }
00188
00189     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00190     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00191     cout << endl << "Duomeniu isskirstymas į kietiakus ir vargsiukus užtruko: " <<
00192         isskirstymo_trukme.count() << " s" << endl;
00193
00194     string failo_pavadinimo_pabaiga = ".txt";
00195     string failo_pavadinimas_kieti = "kietiakai" + to_string(kietiakai.size()) +
00196         failo_pavadinimo_pabaiga;
00197     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
00198         failo_pavadinimo_pabaiga;
00199
00200     studento_vidurkio_skaiciavimas(kietiakai);
00201     studento_vidurkio_skaiciavimas(vargsiukai);
00202     studentu_medianos_skaiciavimas(kietiakai);
00203     studentu_medianos_skaiciavimas(vargsiukai);
00204     duomeniu_surasymas_i_faila(kietiakai, failo_pavadinimas_kieti);
00205     duomeniu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00206 }
00207
00208 template <typename konteineris>
00209 void isrinkVargsiukus(konteineris& visi, konteineris& vargsiukai){
00210     if constexpr (std::is_same_v<konteineris, std::list<Studentas> >){
00211         // Su list trina iš priekio į galą
00212         auto it = visi.begin();
00213         while (it != visi.end()) {
00214             if (it->vidurkis < 5) {
00215                 vargsiukai.push_back(*it);
00216                 it = visi.erase(it);
00217             } else {
00218                 ++it;
00219             }
00220         }
00221     } else {
00222         // Su Vektorius / deque - trina iš galo į priekį, kad mažiau perstumdinėtų
00223         for (auto it = visi.end(); it != visi.begin(); ) {
00224             --it;
00225             if (it->vidurkis < 5) {
00226                 vargsiukai.push_back(*it);
00227                 it = visi.erase(it); // erase grąžina iteratorių į sekantį po ištrinto
00228             }
00229         }
00230     }
00231 }
00232 }
00233 }

```

```

00230 template <typename konteineris>
00231 void strategija_2(){
00232
00233     konteineris visi;
00234     konteineris vargsiukai;
00235
00236     studento_duomenu_skaitymas_is_failo(visi);
00237     studento_vidurkio_skaiciavimas(visi);
00238
00239     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00240
00241     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00242         visi.sort(rikiuoti_pagal_vidurkis);
00243     } else{
00244         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00245     }
00246
00247     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00248     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00249     cout << endl << "Duomenu rikiavimas nuo didziausio vidurkio iki maziausio uztruko: " <<
rikiavimo_trukme.count() << " s" << endl;
00250
00251     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00252
00253     isrinkVargsiukus(visi, vargsiukai);
00254
00255     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00256
00257     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00258         vargsiukai.sort(rikiuoti_pagal_vidurkis);
00259     } else{
00260         sort(vargsiukai.begin(), vargsiukai.end(), rikiuoti_pagal_vidurkis);
00261     }
00262
00263     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00264     cout << endl << "Duomenu isskirstymas i kietiokus ir vargsiukus uztruko: " <<
isskirstymo_trukme.count() << " s" << endl;
00265
00266     string failo_pavadinimo_pabaiga = ".txt";
00267     string failo_pavadinimas_kieti = "kietiakai" + to_string(visi.size()) + failo_pavadinimo_pabaiga;
00268     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
failo_pavadinimo_pabaiga;
00269
00270     studento_vidurkio_skaiciavimas(visi);
00271     studento_vidurkio_skaiciavimas(vargsiukai);
00272     studentu_medianos_skaiciavimas(visi);
00273     studentu_medianos_skaiciavimas(vargsiukai);
00274     duomenu_surasymas_i_faila(visi, failo_pavadinimas_kieti);
00275     duomenu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00276 }
00277
00278 template <typename konteineris>
00279 void strategija_3(){
00280
00281     konteineris visi;
00282     studento_duomenu_skaitymas_is_failo(visi);
00283     studento_vidurkio_skaiciavimas(visi);
00284
00285     auto rikiavimo_pradzia = std::chrono::high_resolution_clock::now();
00286     if constexpr (is_same_v<konteineris, list<Studentas>)> {
00287         visi.sort(rikiuoti_pagal_vidurkis);
00288     } else {
00289         sort(visi.begin(), visi.end(), rikiuoti_pagal_vidurkis);
00290     }
00291     auto rikiavimo_pabaiga = std::chrono::high_resolution_clock::now();
00292     std::chrono::duration<double> rikiavimo_trukme = rikiavimo_pabaiga - rikiavimo_pradzia;
00293     cout << "\nDuomenu rikiavimas nuo didziausio vidurkio iki maziausio uztruko: " <<
rikiavimo_trukme.count() << " s" << endl;
00294
00295     auto isskirstymo_pradzia = std::chrono::high_resolution_clock::now();
00296
00297     // Partitioning students into two groups using stable_partition
00298     auto it = stable_partition(visi.begin(), visi.end(), [](const Studentas& s) {
00299         return s.vidurkis >= 5;
00300     });
00301
00302     konteineris kietiakai(visi.begin(), it);
00303     konteineris vargsiukai(it, visi.end());
00304
00305     auto isskirstymo_pabaiga = std::chrono::high_resolution_clock::now();
00306     std::chrono::duration<double> isskirstymo_trukme = isskirstymo_pabaiga - isskirstymo_pradzia;
00307     cout << "\nDuomenu isskirstymas i kietiokus ir vargsiukus uztruko: " << isskirstymo_trukme.count() <<
"s" << endl;
00308
00309     string failo_pavadinimo_pabaiga = ".txt";
00310     string failo_pavadinimas_kieti = "kietiakai" + to_string(kietiakai.size()) +
failo_pavadinimo_pabaiga;

```



```

00311     string failo_pavadinimas_vargsai = "vargsiukai" + to_string(vargsiukai.size()) +
        failo_pavadinimo_pabaiga;
00312
00313     studento_vidurkio_skaiciavimas(kietiakiai);
00314     studento_vidurkio_skaiciavimas(vargsiukai);
00315     studentu_medianos_skaiciavimas(kietiakiai);
00316     studentu_medianos_skaiciavimas(vargsiukai);
00317     duomenu_surasymas_i_faila(kietiakiai, failo_pavadinimas_kieti);
00318     duomenu_surasymas_i_faila(vargsiukai, failo_pavadinimas_vargsai);
00319 }
00320
00321 #endif

```

## 6.2 manoVektorius.h

```

00001 #ifndef MANOVEKTORIUS_H
00002 #define MANOVEKTORIUS_H
00003 #include "pagalbines.h"
00004
00005 template <typename ElementoTipas>
00006 class Vektorius{
00007
00008 private:
00009     ElementoTipas* masyvas;
00010     size_t dydis;
00011     size_t talpa;
00012
00013     void reallocate(size_t naujaTalpa);
00014
00015 public:
00016     // The rule of five
00017     Vektorius();
00018     ~Vektorius();
00019
00020     Vektorius(const Vektorius& pagrindinis); // Copy constructor
00021     Vektorius& operator=(const Vektorius& pagrindinis); // Copy assignment operator
00022     Vektorius(Vektorius&& pagrindinis) noexcept; // Move constructor
00023     Vektorius& operator=(Vektorius&& pagrindinis) noexcept; // Move assignment operator
00024     Vektorius(ElementoTipas* first, ElementoTipas* last);
00025
00026     // operators
00027     ElementoTipas& operator[](size_t index);
00028     const ElementoTipas& operator[](size_t index) const;
00029
00030     // getters
00031     size_t size() const;
00032     size_t capacity() const;
00033
00034     // itterators
00035     ElementoTipas* begin() { return masyvas; }
00036     ElementoTipas* end() { return masyvas + dydis; }
00037
00038     const ElementoTipas* begin() const { return masyvas; }
00039     const ElementoTipas* end() const { return masyvas + dydis; }
00040
00041     ElementoTipas& front() { return masyvas[0]; }
00042     ElementoTipas& back() { return masyvas[dydis - 1]; }
00043
00044     const ElementoTipas& front() const { return masyvas[0]; }
00045     const ElementoTipas& back() const { return masyvas[dydis - 1]; }
00046
00047     void push_back(const ElementoTipas& elementas);
00048     void pop_back();
00049     bool empty() const { return dydis == 0; }
00050     ElementoTipas& at(size_t index);
00051     void clear();
00052     void shrink_to_fit();
00053     void reserve(size_t naujaTalpa);
00054     void erase(size_t index);
00055     ElementoTipas* erase(ElementoTipas* it);
00056
00057 };
00058
00059
00060
00061 template <typename ElementoTipas>
00062 Vektorius<ElementoTipas>::Vektorius() {
00063     talpa = 10;
00064     dydis = 0;
00065     masyvas = new ElementoTipas[talpa];
00066 }
00067
00068 template <typename ElementoTipas>
00069 Vektorius<ElementoTipas>::~~Vektorius() {

```

```

00070     delete[] masyvas;
00071 }
00072
00073 template <typename ElementoTipas>
00074 Vektorius<ElementoTipas>::Vektorius(const Vektorius& pagrindinis){
00075     dydis = pagrindinis.dydis;
00076     talpa = pagrindinis.talpa;
00077     masyvas = new ElementoTipas[talpa];
00078
00079     for (size_t i = 0; i < dydis; ++i) {
00080         masyvas[i] = pagrindinis.masyvas[i];
00081     }
00082 }
00083
00084 // Copy assignment operator
00085 template <typename ElementoTipas>
00086 Vektorius<ElementoTipas>& Vektorius<ElementoTipas>::operator=(const Vektorius& pagrindinis) {
00087     if (this == &pagrindinis) return *this; // Self-assignment check
00088
00089     delete[] masyvas; // Free existing memory
00090
00091     dydis = pagrindinis.dydis;
00092     talpa = pagrindinis.talpa;
00093     masyvas = new ElementoTipas[talpa];
00094
00095     for (size_t i = 0; i < dydis; ++i) {
00096         masyvas[i] = pagrindinis.masyvas[i];
00097     }
00098
00099     return *this;
00100 }
00101
00102 // Move constructor
00103 template <typename ElementoTipas>
00104 Vektorius<ElementoTipas>::Vektorius(Vektorius&& pagrindinis) noexcept {
00105     talpa = pagrindinis.talpa;
00106     dydis = pagrindinis.dydis;
00107     masyvas = pagrindinis.masyvas; // naujo masyvo rodykle yra pagrindinio rodykle.
00108
00109     pagrindinis.masyvas = nullptr; // pagrindine rodykle neberodo i nieka.
00110     pagrindinis.dydis = 0;
00111     pagrindinis.talpa = 0;
00112 }
00113
00114 // Move assignment operator
00115 template <typename ElementoTipas>
00116 Vektorius<ElementoTipas>& Vektorius<ElementoTipas>::operator=(Vektorius&& pagrindinis) noexcept {
00117     if (this == &pagrindinis) return *this; // Self-assignment check
00118
00119     delete[] masyvas; // Release the current memory
00120
00121     masyvas = pagrindinis.masyvas; // Take ownership of the pagrindinis vector's data
00122     dydis = pagrindinis.dydis;
00123     talpa = pagrindinis.talpa;
00124
00125     pagrindinis.masyvas = nullptr; // Reset the pagrindinis vector
00126     pagrindinis.dydis = 0;
00127     pagrindinis.talpa = 0;
00128
00129     return *this;
00130 }
00131
00132 template <typename ElementoTipas>
00133 size_t Vektorius<ElementoTipas>::size() const {
00134     return dydis;
00135 }
00136
00137 template <typename ElementoTipas>
00138 size_t Vektorius<ElementoTipas>::capacity() const {
00139     return talpa;
00140 }
00141
00142 template <typename ElementoTipas>
00143 void Vektorius<ElementoTipas>::reallocate(size_t naujaTalpa) {
00144     ElementoTipas* newArray = new ElementoTipas[naujaTalpa];
00145
00146     for (size_t i = 0; i < dydis; ++i) {
00147         newArray[i] = masyvas[i];
00148     }
00149
00150     delete[] masyvas;
00151
00152     masyvas = newArray;
00153     talpa = naujaTalpa;
00154 }
00155
00156 template <typename ElementoTipas>

```

```

00157 void Vektorius<ElementoTipas>::push_back(const ElementoTipas& elementas) {
00158     if(dydis == talpa)
00159     {
00160         reallocate(talpa * 2); // padvigubina dabartine talpa
00161     }
00162
00163     masyvas[dydis] = elementas;
00164     ++dydis;
00165 }
00166
00167
00168 template <typename ElementoTipas>
00169 void Vektorius<ElementoTipas>::pop_back() {
00170     if (dydis > 0) {
00171         --dydis;
00172     }
00173 }
00174
00175 template <typename ElementoTipas>
00176 ElementoTipas& Vektorius<ElementoTipas>::operator[](size_t index) {
00177     return masyvas[index];
00178 }
00179
00180 template <typename ElementoTipas>
00181 const ElementoTipas& Vektorius<ElementoTipas>::operator[](size_t index) const {
00182     return masyvas[index];
00183 }
00184
00185 template <typename ElementoTipas>
00186 ElementoTipas& Vektorius<ElementoTipas>::at(size_t index) {
00187     if (index >= dydis) {
00188         throw out_of_range("Index is out of range.");
00189     }
00190
00191     return masyvas[index];
00192 }
00193
00194
00195 template <typename ElementoTipas>
00196 void Vektorius<ElementoTipas>::clear() {
00197     if constexpr (!std::is_trivially_destructible<ElementoTipas>::value) {
00198         for (size_t i = 0; i < dydis; ++i) {
00199             masyvas[i].~ElementoTipas(); // aunaikina kiekviena elementa
00200         }
00201     }
00202
00203     dydis = 0;
00204 }
00205
00206 template <typename ElementoTipas>
00207 void Vektorius<ElementoTipas>::shrink_to_fit() {
00208     if (dydis < talpa) {
00209         reallocate(dydis);
00210     }
00211 }
00212
00213 template <typename ElementoTipas>
00214 void Vektorius<ElementoTipas>::reserve(size_t naujaTalpa) {
00215     if (naujaTalpa > talpa) {
00216         reallocate(naujaTalpa);
00217     }
00218 }
00219
00220 template <typename ElementoTipas>
00221 void Vektorius<ElementoTipas>::erase(size_t index) {
00222     if (index >= dydis) {
00223         throw out_of_range("Index out of range in erase()");
00224     }
00225
00226     if constexpr (!std::is_trivially_destructible<ElementoTipas>::value) {
00227         masyvas[index].~ElementoTipas(); // sunaikina istrinta elementa jei reikia
00228     }
00229
00230     // Visus elementus perstumia i kaite puse nuo index+1 iki galo
00231     for (size_t i = index; i < dydis - 1; ++i) {
00232         masyvas[i] = move(masyvas[i + 1]);
00233     }
00234
00235     --dydis;
00236 }
00237
00238 template <typename ElementoTipas>
00239 bool operator==(const Vektorius<ElementoTipas>& lhs, const Vektorius<ElementoTipas>& rhs) {
00240     if (lhs.size() != rhs.size()) return false;
00241
00242     for (size_t i = 0; i < lhs.size(); ++i) {
00243         if (lhs[i] != rhs[i]) return false;

```

```

00244     }
00245
00246     return true;
00247 }
00248
00249 template <typename ElementoTipas>
00250 bool operator!=(const Vektorius<ElementoTipas>& lhs, const Vektorius<ElementoTipas>& rhs) {
00251     return !(lhs == rhs);
00252 }
00253
00254 template <typename ElementoTipas>
00255 bool operator<(const Vektorius<ElementoTipas>& lhs, const Vektorius<ElementoTipas>& rhs) {
00256     size_t minSize = (lhs.size() < rhs.size()) ? lhs.size() : rhs.size();
00257
00258     for (size_t i = 0; i < minSize; ++i) {
00259         if (lhs[i] < rhs[i]) return true;
00260         if (lhs[i] > rhs[i]) return false;
00261     }
00262
00263     return lhs.size() < rhs.size();
00264 }
00265
00266 template <typename ElementoTipas>
00267 bool operator>(const Vektorius<ElementoTipas>& lhs, const Vektorius<ElementoTipas>& rhs) {
00268     return rhs < lhs;
00269 }
00270
00271 template <typename ElementoTipas>
00272 Vektorius<ElementoTipas>::Vektorius(ElementoTipas* pradzia, ElementoTipas* pabaiga) {
00273     dydis = pabaiga - pradzia;
00274     talpa = dydis;
00275     masyvas = new ElementoTipas[talpa];
00276     for (size_t i = 0; i < dydis; ++i) {
00277         masyvas[i] = pradzia[i];
00278     }
00279 }
00280
00281 template <typename ElementoTipas>
00282 ElementoTipas* Vektorius<ElementoTipas>::erase(ElementoTipas* it) {
00283     size_t index = it - masyvas;
00284     erase(index);
00285     return masyvas + index;
00286 }
00287 #endif

```

## 6.3 pagalbines.h

```

00001 #include <iostream>
00002 #include <iomanip>
00003 #include <vector>
00004 #include <fstream>
00005 #include <string>
00006 #include <algorithm>
00007 #include <limits>
00008 #include <stdlib.h>
00009 #include <ctime>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <functional>
00013 #include <cctype>
00014 #include <chrono>
00015 #include <list>
00016 #include <deque>
00017 #include <cmath>
00018 #include <iterator>
00019 #include <type_traits>
00020 #include <cstdint>
00021 #include <stdexcept>
00022
00023
00024 using std:: cout;
00025 using std:: cin;
00026 using std:: endl;
00027 using std:: string;
00028 using std:: vector;
00029 using std:: setw;
00030 using std:: left;
00031 using std:: right;
00032 using std:: setprecision;
00033 using std:: fixed;
00034 using std:: sort;
00035 using std:: numeric_limits;
00036 using std:: streamsize;
00037 using std:: srand;

```

```
00038 using std:: rand;
00039 using std:: time;
00040 using std:: ifstream;
00041 using std:: ofstream;
00042 using std:: getline;
00043 using std:: stringstream;
00044 using std:: greater;
00045 using std:: any_of;
00046 using :: isdigit;
00047 using std:: atoi;
00048 using :: tolower;
00049 using std:: transform;
00050 using std:: invalid_argument;
00051 using std:: runtime_error;
00052 using std:: exception;
00053 using std:: to_string;
00054 using std:: list;
00055 using std:: deque;
00056 using std:: is_same_v;
00057 using std:: move;
00058 using std:: ostream;
00059 using std:: istream;
00060 using std:: chrono:: round;
00061 using std:: out_of_range;
00062 using std:: stable_partition;
```



# Index

prisistatymas

Studentas, [15](#)

Projekto paleidimo instrukcija, [1](#)

Studentas, [15](#)

prisistatymas, [15](#)

Vektorius< ElementoTipas >, [16](#)

Zmogus, [16](#)